

* WS와 WAS의 차이점.

1. WS (Web Server, 웹서버)

클라이언트(브라우저)의 요청을 받아 **정적인** 자원(HTML, CSS, JS, 이미지 등)을 응답해주는 서버. 예) Apache, Nginx

2. WAS (Web Application Server)

정적자원뿐만 아니라 **동적인 처리**(DB 조회, 비즈니스 로직 실행 후 결과 생성)가 가능한 서버.

내부에 **서블릿 컨테이너** (Servlet Container)를 가지고 있어 Servlet, JSP 코드 실행이 가능하다. 예) Tomcat, JBoss, WebLogic

❖ Servlet은 단독으로 동작하는 것이 아니라 반드시 WAS(서블릿 컨테이너)위에서만 동작한다.

⇒ 개발환경 구축 시 반드시 Tomcat 설치 및 연동이 선행되어야 함!

3. 결론

둘 다 클라이언트(브라우저)의 요청에 응답하는 서버이지만, 정적인 자원으로만 응답이 가능한 WS와 달리 WAS는 서블릿 컨테이너를 통해 동적인 처리까지 가능하다.

* 서블릿(Servlet)이란?

↳ '작음'을 의미하는 접미사

Server + Applet (= Server + (Application + -let))

Java를 이용하여 사용자의 요청을 받아 처리하고 그 결과를 클라이언트에게 전송하는 **서버측 자바 클래스**

1. 서블릿의 역할

WAS가 클라이언트의 요청을 받고 처리해야 할 아래 항목 중 공통 HTTP 스펙 처리 과정에 해당하는

1, 2, 4, 5를 서블릿 컨테이너가 대신 처리하므로 개발자는 요청정보인 `HttpServletRequest`와 응답정보인 `HttpServletResponse` 객체를 활용하여 비즈니스 로직 구현에만 집중할 수 있다.

1) 서버 TCP/IP 연결 대기, 소켓 연결

2) HTTP 요청 메시지 파싱 (HTTP 메서드, URL, Header, Body 파싱)

3) 비즈니스 로직 실행 및 DB연동 (실질적인 애플리케이션 핵심 로직)

4) HTTP 응답 메시지 생성

5) TCP/IP 응답 전달 및 소켓 대기.

2. 서블릿 기본 구조

1) ①단계 : 어노테이션

컴파일러 (또는 서버)에게 전달하는 특별한 구성. 어노테이션마다 컴파일러/서버가 자동으로 특정 동작 수행.

② WebServlet ("요청받을주소"): 이 서블릿이 요청을 받아 처리할 주소를 매핑(연결)

↳ web.xml 파일 없이 어노테이션 하나로 URL 매핑 설정 가능!

2) HttpServlet 클래스 상속을 받아야 서블릿 컨테이너(Tomcat)가 이 클래스를 서블릿으로 인식하고

이미 정의된 `doGet()`, `doPost()` 메서드를 원하는 동작에 맞게 오버라이딩(재정의)하여 사용 가능.

* Spring

1. Spring : Java 기반 애플리케이션 개발을 지원하는 프레임워크. POJO 기반으로 객체를 생성 및 관리함.

2. Spring의 주요 특징

1) POJO (Plain Old Java Object) **순수한 일반 자바 객체**

특수한 클래스가 아닌 일반 자바 객체로 비즈니스 로직을 작성할 수 있게 한다.

(유연하고 테스트하기 쉬운 코드를 지향하기 위함!)

★ POJO의 철학을 지향할 뿐, 개발의 편의성을 위해 어노테이션 기반 기능 추가도 허용한다.

2) IoC (Inversion of Control, 제어의 역전)

객체의 생성과 생명주기 관리 책임을 개발자가 아닌 프레임워크에 위임하여 애플리케이션의 구조적 일관성과 확장성을 확보한다. (결합도를 낮추어 유연하고 테스트하기 쉬운 코드를 만들기 위함!)

↳ new 키워드 사용 없이 Spring이 필요한 객체를 Spring이 대신 생성하여 주입

3) DI (Dependency Injection, 의존성 주입)

IoC 개념을 구현하는 구체적인 패턴으로, 필요한 객체를 직접 생산하는 대신 **클래스 외부에서 주입받아 사용하는 방식**. 결합도를 낮추고 테스트 용이성을 극대화한다.

- 생성자 주입 : 생성자를 통해 객체를 주입받는 방식.

- 필드 주입 (@Autowired) : 필드에 직접 어노테이션을 부여하여 주입하는 방식

- Setter 주입 : Setter 메서드를 통해 주입받는 방식.

```
② Controller    /** 생성자 주입 */
public class ExController {
    private final ExService ex;

    @Autowired    //생략가능
    public ExController(ExService ex) {
        this.ex = ex;
    }
}
```

```
② Controller    /** 필드 주입 */
public class ExController {
    @Autowired
    private final ExService ex;
}
```

```
② Controller    /** Setter 주입 */
public class ExController {
    private final ExService ex;

    @Autowired
    public void setService(ExService ex) {
        this.ex = ex;
    }
}
```

4) AOP (Aspect-Oriented Programming)

공통적으로 반복되는 코드를 분리하여 별도로 관리하는 프로그래밍 방식.

해당 비즈니스 로직에 집중할 수 있도록 공통 기능을 분리하여 처리하고, 이를 통해 **관심사 분리**

(Separation of Concerns)를 실현한다. ↳ 로그, 보안, 트랜잭션 등.

공통 기능을 별도 클래스로 분리하면 Spring이 지정된 서비스 클래스의 메서드 실행 시점에 공통 기능을 자동 삽입해준다.